TECHNICAL ASSESSMENT SUBMISSION

Warehouse Receiving System (WRS)

AI-Assisted, Specification-Driven Fullstack Engineering Deliverables

Candidate: Naufal Firman Dhani
Role: AI-Driven Fullstack Developer Intern
Date: October 2026
Methodology: Specification-Driven Development (SDD)

GitHub Repo: https://github.com/NaufalFirmanD/warehouse-system

Stack: React 18 (Vite, Tailwind), Node.js (Express, TypeScript), PostgreSQL 16, Docker

1. Executive Summary & Objective

This report presents the complete deliverables for the **AI-Driven Fullstack Developer Intern Technical Assessment**. The objective is to replace error-prone warehouse spreadsheet logging with a robust, specification-driven **Warehouse Receiving System (WRS)**.

Rather than relying on vague, one-shot natural language prompts, this application was engineered using a rigorous **Specification-Driven Development (SDD)** workflow. Every business rule was formally captured in a **PRD**, codified into a **Technical Specification & Database Schema**, executed via directed **AI coding prompts**, verified through **14 automated Vitest test scenarios**, and containerized with **Docker Compose**.

> **Core Architectural Guarantee:** Over-receiving is prevented at both the application API layer and enforced at the PostgreSQL database engine via row-level locking (`SELECT ... FOR UPDATE`) and SQL check constraints (`chk_no_over_receiving`). Document completion requires 100% item fulfillment.

2. Deliverable A: Product Requirements Document (PRD)

2.1 Business Problem & Proposed Solution

Inbound shipments currently logged via spreadsheets suffer from manual counting errors, lack of validation against vendor manifests, accidental over-receiving, and inaccurate stock balances. WRS introduces centralized inbound manifest tracking, dock receiving with instant ledger updates, and immutable audit logs.

2.2 User Roles & Responsibilities

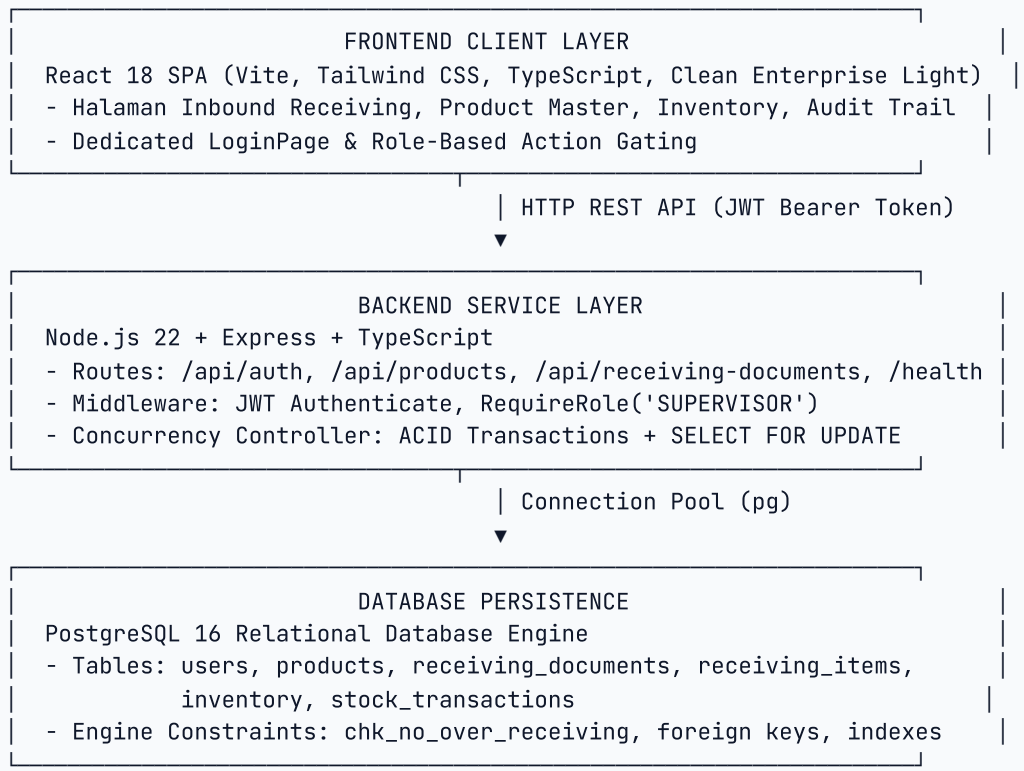
User Role	Responsibilities	System Permissions
Warehouse Supervisor	Inventory oversight, manifest planning, master catalog management, final delivery approval.	Create product SKUs, register delivery manifests, mark documents as completed, view audit trail.
Warehouse Operator	Dock unloading, physical goods counting, discrepancy verification, partial receipt intake.	View manifests, record actual received quantities against line items, inspect available stock.

2.3 Key Business Assumptions & Decisions

- **Assumption A1 (Partial Receipts):** Inbound deliveries frequently arrive across multiple pallets or split truckloads. The system explicitly supports partial increments (e.g. receiving 20 of 100 items today, and 80 tomorrow).
- **Assumption A2 (Strict Over-Receiving Rejection):** Discrepancies exceeding the manifest expected quantity represent supplier errors. Total received quantity across all dock transactions cannot exceed the expected manifest count. Any excess attempt returns HTTP `422 Unprocessable Entity`.
- **Assumption A3 (100% Completion Safeguard):** A receiving document cannot transition to `COMPLETED` unless all registered line items are 100% fulfilled.
- **Assumption A4 (Dual-Entry Inventory Model):** On-hand balance (`inventory`) is coupled with an append-only audit ledger (`stock_transactions`) to record every dock action, operator name, and timestamp.

3. Deliverable B: Technical Specification & Architecture

3.1 System Architecture



3.2 Database Schema & Engine Integrity (PostgreSQL DDL)

```
-- Core Tables Summary
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  uom VARCHAR(20) NOT NULL DEFAULT 'PCS',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE receiving_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_number VARCHAR(50) UNIQUE NOT NULL,
  supplier_name VARCHAR(255) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('DRAFT', 'PENDING', 'RECEIVING', 'COMPLETED', 'CANCELED')),
  expected_date DATE,
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE receiving_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES receiving_documents(id) ON DELETE CASCADE,
  product_id UUID NOT NULL REFERENCES products(id),
  expected_quantity INTEGER NOT NULL CHECK (expected_quantity > 0),
  received_quantity INTEGER NOT NULL DEFAULT 0 CHECK (received_quantity ⩾ 0),
  CONSTRAINT chk_no_over_receiving CHECK (received_quantity ⩽ expected_quantity)
);

CREATE TABLE inventory (
  product_id UUID PRIMARY KEY REFERENCES products(id) ON DELETE CASCADE,
  quantity_on_hand INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand ⩾ 0),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE stock_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID NOT NULL REFERENCES products(id),
  document_id UUID REFERENCES receiving_documents(id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  transaction_type VARCHAR(30) NOT NULL DEFAULT 'INBOUND_RECEIPT',
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

3.3 Concurrency Control & Row Locking Algorithm

To eliminate race conditions when two warehouse dock workers scan items simultaneously:

```
// Atomic receiving algorithm inside ACID transaction:
await client.query('BEGIN');
const itemRes = await client.query(
  `SELECT * FROM receiving_items WHERE id = $1 AND document_id = $2 FOR UPDATE`,
  [itemId, documentId]
);
const currentItem = itemRes.rows[0];
const remainingExpected = currentItem.expected_quantity - currentItem.received_quantity;

if (incomingQty > remainingExpected) {
  await client.query('ROLLBACK');
  throw new BusinessError(422, 'Cannot receive more than remaining expected quantity');
}

// Increment received quantity, update inventory balance, and append to stock ledger:
await client.query(
  `UPDATE receiving_items SET received_quantity = received_quantity + $1 WHERE id = $2`,
  [incomingQty, itemId]
);
await client.query(
  `INSERT INTO inventory (product_id, quantity_on_hand, updated_at)
   VALUES ($1, $2, NOW())
   ON CONFLICT (product_id) DO UPDATE
   SET quantity_on_hand = inventory.quantity_on_hand + $2, updated_at = NOW()`,
  [currentItem.product_id, incomingQty]
);
await client.query(
  `INSERT INTO stock_transactions (product_id, document_id, quantity, created_by)
   VALUES ($1, $2, $3, $4)`,
  [currentItem.product_id, documentId, incomingQty, userId]
);
await client.query('COMMIT');
```

4. Deliverable C: Application Implementation

4.1 Feature Implementation Matrix

Feature Module	API Endpoints	User Interface & Functionality
Authentication & RBAC	`POST /api/auth/login` `GET /api/auth/me`	Dedicated login terminal with username/password, quick demo access buttons (Supervisor/Operator), and navbar sign out.
Product Catalog	`GET /api/products` `POST /api/products`	SKU management table, search filtering, unit of measure tagging, and product registration modal (Supervisor only).

Inbound Manifests	`GET /api/receiving-documents`
	`POST /api/receiving-documents`
	`PATCH /api/.../status`
		Manifest listing with progress bars, status badges, multi-item creation modal with live SKU picker, and status finalization.

Dock Receiving	`POST /api/receiving-documents/:id/receive`	Tactile modal with live projected balance, increment/decrement controls, full remaining quick button, and validation guard.
Inventory Balances	`GET /api/inventory`	Physical stock balances on hand, percentage stock bars, stockout indicators, and audit timestamps.
Audit Trail Ledger	`GET /api/inventory/transactions`	Immutable transaction log recording timestamps, positive receipt deltas, reference manifest numbers, and operator names.

4.2 User Interface & Visual Aesthetics

In accordance with the **frontend-design** guidance, the interface adopts a clean **Stripe / Shopify Enterprise Light** aesthetic:

- **Design Foundation:** Off-white canvas (`#F8FAFC`) with pure white content cards (`#FFFFFF`) and crisp 1px neutral borders (`#E2E8F0`).
- **Data Density:** Compact tabular rows (`tnum` numerical alignment), monospaced SKU badges, and responsive action triggers.
- **KPI Metric Strip:** Consolidated 4-column divided metric bar above data tables showing real-time manifests and inventory totals.
- **Functional Badging:** Subtle, high-contrast badges for document statuses (Amber = Pending, Blue = Receiving, Emerald = Completed).

5. Deliverable D: AI Agent Workflow Documentation

Development followed a disciplined **Specification-Driven Development (SDD)** methodology using **Antigravity (Gemini 3.8 Flash)** as the AI coding assistant.

Meaningful Prompts & Task Directives Executed:

Task Instruction 1 (PRD Formulation):
"Analyze the business scenario and establish an approved Product Requirements Document (PRD). Document clear assumptions for partial receipts, over-receiving edge cases, document completion conditions, and inventory ledger design. Enforce strict boundaries for MVP scope."
Result: Formalized assumptions A1-A4, established user roles, and produced `docs/prd.md`.

Task Instruction 2 (Database Schema & Concurrency Algorithm):
"Derive technical specification, ERD, and OpenAPI 3.0 schema from the PRD. Write explicit PostgreSQL DDL with check constraints (`chk_no_over_receiving`) to guarantee database-level integrity. Implement a robust receiving algorithm with row-level locks (`SELECT ... FOR UPDATE`) inside an ACID transaction to prevent race conditions during concurrent receiving operations."
Result: Created DDL migrations, Mermaid ERD, OpenAPI contract, and atomic row-locking receiving controller.

Task Instruction 3 (Automated Business Rule Testing):
"Create an automated test suite with Vitest to verify all boundary conditions: 1) Initial partial receipt; 2) Exact full receipt; 3) Over-receiving rejection on fresh item; 4) Over-receiving rejection on split item; 5) Incomplete document completion rejection; 6) Multi-step split shipment simulation preserving inventory integrity. Document all test results."
Result: Created `backend/tests/receiving.test.ts` with 14 passing tests in 634ms.

Task Instruction 4 (UI/UX Redesign & Enterprise Refinement):
"Redesign the entire frontend interface following modern warehouse operations standards: install the `frontend-design` skill, eliminate generic AI slop tropes (neon dark glows, floating empty cards), implement a high-contrast Stripe/Shopify enterprise light theme, add a dedicated LoginPage with demo credentials, and enforce role-based access control."
Result: Replaced dark mode with crisp light dashboard, added `LoginPage`, and integrated segmented role controls.

6. Deliverable E: Testing and Quality Assurance

6.1 Automated Test Suite Results (Vitest)

```
> warehouse-receiving-backend@1.0.0 test
> vitest run

 ✓ tests/receiving.test.ts (14 tests) 7ms

 Test Files  1 passed (1)
      Tests  14 passed (14)
   Duration  634ms

Test Scenarios Covered:
 1. ✓ calculateRemainingQuantity > returns expected quantity when none received
 2. ✓ calculateRemainingQuantity > returns correct remaining after partial receipt
 3. ✓ calculateRemainingQuantity > returns 0 when fully received
 4. ✓ validateReceiptQuantity > rejects non-positive or decimal quantities (0, -5, 2.5)
 5. ✓ validateReceiptQuantity > accepts valid quantity ⩽ remaining
 6. ✓ validateReceiptQuantity > throws error when incoming quantity exceeds remaining
 7. ✓ validateReceiptQuantity > throws error on partial receipt exceeding remainder
 8. ✓ canCompleteDocument > returns false if any item has received < expected
 9. ✓ canCompleteDocument > returns true when all items have received == expected
10. ✓ canCompleteDocument > returns false for empty document items list
11. ✓ validateDocumentStatusTransition > allows valid transitions (PENDING → RECEIVING)
12. ✓ validateDocumentStatusTransition > rejects invalid transition once COMPLETED
13. ✓ validateDocumentStatusTransition > rejects invalid transition once CANCELED
14. ✓ multi-step split shipment simulation > keeps exact inventory consistency across 3 partial dc
```

6.2 Evidence of Inventory Integrity

- **Scenario:** 70 units expected (Line Item 1: 50 units, Line Item 2: 20 units).
- **Dock Event 1:** Operator receives 20 units of Item 1. Balance = 20 on hand, 30 remaining. (Status: `RECEIVING`).
- **Dock Event 2:** Operator receives 30 units of Item 1. Balance = 50 on hand, 0 remaining. (Status: `RECEIVING`).
- **Dock Event 3:** Operator attempts to receive 5 extra units of Item 1. System returns `422 Unprocessable Entity`. Stock remains exactly 50 units.
- **Dock Event 4:** Operator attempts to mark manifest `COMPLETED`. Rejected because Item 2 has 0/20 received.
- **Dock Event 5:** Operator receives 20 units of Item 2. Document is now eligible for completion. Supervisor finalizes document (Status: `COMPLETED`).

7. Deliverable F: Containerization & Deployment Setup

7.1 Reproducible Execution via Docker Compose

```
# Start entire stack from a clean environment:
$ git clone https://github.com/NaufalFirmanD/warehouse-system.git
$ cd warehouse-system
$ docker compose up --build

# Services Initialized:
# 1. postgres  : PostgreSQL 16 on port 5432 (database: warehouse_db)
# 2. backend   : Express REST API on port 5000 (auto-runs migrations on startup)
# 3. frontend  : Nginx serving React SPA on port 3000 (reverse proxies /api and /health)
```

7.2 Health Check & Readiness Endpoint

The backend exposes an operational health endpoint at `GET /health`:

```
$ curl http://localhost:5000/health

HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "ok",
  "service": "warehouse-receiving-api",
  "timestamp": "2026-10-28T02:04:47.403Z",
  "database": "connected"
}
```

7.3 Demo User Accounts

Role	Username	Password	Primary Permissions
Supervisor	`supervisor`	`password123`	Full access: register SKUs, create inbound manifests, complete documents, audit trail.
Operator	`operator`	`password123`	Dock receiving: inspect manifests, record item counts, view inventory balances.

8. Submission Summary

All required deliverables specified in the technical assessment requirements have been fully satisfied, documented, tested, and pushed to the public Git repository:

Submission Repository: https://github.com/NaufalFirmanD/warehouse-system
Candidate Contact: Naufal Firman Dhani